

SMARTFLOW BEAUTY

Развёртывание

Для владельца установки и технического специалиста

MVP • Пилотное тестирование • 2026

Версия документа: 1.0

Для кого: владелец установки или технический специалист

Важно: сначала выполняйте установку на тестовых ботах и тестовом календаре.

1. Что разворачивается

Одна установка SmartFlow использует одну Google-таблицу, один связанный Apps Script проект, два разных Telegram-бота и один или несколько Google Calendar. Один Web App принимает обновления обоих ботов: `?bot=client` для Client Bot и `?bot=admin` для Admin Bot.

2. Что подготовить заранее

- Google-аккаунт владельца установки;
- два новых бота, созданных через официального [@BotFather](#);
- Telegram ID первого администратора;
- календарь для каждого мастера либо явно согласованный общий календарь;
- локальный Git-репозиторий SmartFlow;
- Node.js и [@google/clasp](#), если код загружается из репозитория;
- чистый клиентский шаблон `outputs/smartflow-client-template/SmartFlow Beauty Client Template.xlsx`.

Никому не отправляйте токены в чатах. Потерянный токен следует немедленно перевыпустить через BotFather.

Если основной телефон — iPhone

Apple ID и Google Account — разные учётные записи. Apple ID нужен для App Store и служб Apple, а доступ к Google Sheets, Apps Script и Google Calendar выдаётся на конкретный адрес Google Account. Заранее запишите, какой именно Google-аккаунт является владельцем установки.

Первичное развёртывание выполняйте на компьютере. На iPhone неудобно и ненадёжно создавать Apps Script deployment, управлять триггерами и запускать `clasp`; самого iPhone для полной установки недостаточно. Телефон подходит для Telegram, ежедневной проверки календаря и таблицы.

Для работы с iPhone:

1. Установите из App Store актуальные Telegram, Google Drive, Google Sheets и Google Calendar.
2. Войдите в Google-приложения под аккаунтом владельца установки либо под отдельным рабочим Google-аккаунтом, которому выданы нужные права.
3. Если на телефоне несколько Google-аккаунтов, проверяйте адрес под аватаром отдельно в Sheets и Calendar: выбранные аккаунты могут отличаться.
4. Откройте ссылку на таблицу и примите приглашение. Доступ Telegram-администратора к боту сам по себе не даёт доступа к Google-таблице или календарю.
5. В Google Calendar откройте меню и убедитесь, что рабочие календари включены и отображаются. При желании их можно подключить к стандартному приложению «Календарь» iPhone через настройки аккаунтов, но для диагностики предпочтительно приложение Google Calendar.
6. В iOS откройте «Настройки → Приложения → Telegram → Уведомления», включите уведомления, звук и нужные способы показа. Проверьте также режим «Фокусирование»: он

может задерживать уведомления.

7. Настройте резервные способы двухэтапной проверки Google Account. Не храните токены ботов в заметках, синхронизируемых через iCloud, и на скриншотах; используйте менеджер паролей.

Если ссылка Google открывается в Safari с сообщением об отсутствии доступа, сначала проверьте аватар и адрес активного Google-аккаунта. Выйдите из неверного аккаунта или переключитесь на тот адрес, которому предоставлен доступ.

3. Создание Telegram-ботов

1. Откройте [@BotFather](#) и выполните `/newbot`.
2. Создайте Client Bot с понятным клиентским именем и уникальным username.
3. Сохраните выданный токен во временном защищённом месте.
4. Повторите шаги для отдельного Admin Bot.
5. Настройте описание, фотографию и команды только после базовой проверки.
6. Не добавляйте ботов в группы: текущий интерфейс рассчитан на личный диалог.

4. Подготовка Google Sheets

1. Загрузите шаблон XLSX в Google Drive и откройте как Google Sheets либо создайте копию эталонной таблицы.
2. Убедитесь, что имена листов сохранены: Settings, Messages, UserStates, UserSessions, Customers, CustomerProfiles, CustomerVisitHistory, CustomerConflicts, CustomerServiceSettings, Locations, Providers, ProviderSchedule, ProviderScheduleOverrides, Services, Requests, RequestOptions, RequestRecipients, Appointments, AuditLog и WeekDays.
3. Не переименовывайте листы и заголовки колонок вручную.
4. Предоставьте редактирование только владельцу и доверенному техническому специалисту. Администраторам Telegram не обязательно давать доступ к таблице.
5. Установите для столбца значений `AdminTelegramIds` текстовый формат. Telegram ID нельзя хранить как число с округлением.

5. Начальное заполнение Settings

В столбце значения заполните:

- `Language`: `uk`, `ru` или `en`;
- `TimeZone`: например `Europe/Kyiv`;
- `AppScriptUrl`: пока оставьте пустым, он появится после deployment;
- `AdminTelegramIds`: Telegram ID первого администратора, несколько ID — через запятую без пробелов;
- `ReminderDayBefore`: `TRUE` или `FALSE`;
- `DefaultCalendarId`: Calendar ID для безопасного fallback, обычно `primary` только если это действительно нужный календарь;
- `DefaultWorkStartTime`, `DefaultWorkEndTime`: рабочий интервал по умолчанию;
- `BookingDaysAhead`: от 1 до 365;
- `PaginationPageSize`: от 1 до 10, рекомендуемое значение 5.

Токены не вводятся в Settings. Откройте Apps Script → Project Settings → Script Properties и создайте `SMARTFLOW_CLIENT_BOT_TOKEN` и `SMARTFLOW_ADMIN_BOT_TOKEN`. В значения вставьте

соответствующие токены из BotFather и сохраните свойства. Не записывайте токены в таблицу, AuditLog или журнал установки.

Часовой пояс Apps Script, Settings и календарей должен совпадать. Несовпадение приводит к неверным слотам и напоминаниям.

6. Создание связанного Apps Script проекта

Самый простой путь — открыть Google-таблицу и выбрать «Расширения → Apps Script». Проект станет связанным с таблицей и `SpreadsheetApp.getActiveSpreadsheet()` будет обращаться к правильной базе.

Откройте Project Settings и скопируйте Script ID.

Загрузка через clasp

1. Установите clasp: `npm install -g @google/clasp`.
2. Авторизуйтесь: `clasp login`.
3. В каталоге `AppsScript` создайте локальный `.clasp.json` со значениями `scriptId` нового проекта и `rootDir` равным пустой строке.
4. Убедитесь, что `.clasp.json` не отслеживается Git.
5. Выполните `clasp push` из каталога `AppsScript`.
6. Если PowerShell блокирует `clasp.ps1`, используйте `clasp.cmd push` либо разрешите подписанные локальные скрипты согласно политике вашей организации. Не отключайте Execution Policy глобально без необходимости.
7. Откройте Apps Script и убедитесь, что загружены `.gs` файлы и `appsscript.json`.

Важно: команда называется `clasp`, не `clash`.

Безопасная загрузка в отдельный проект клиента

Для production-установок используйте клиентский профиль и утилиту из корня репозитория. Например, первый профиль находится в `deployments/salon-alice`. Скопируйте `deployment.example.json` в игнорируемый Git файл `deployment.local.json` и замените плейсхолдер реальным Script ID клиента. Токены Telegram в профиль не добавляются.

Сначала выполните проверку без загрузки:

```
.\Scripts\deploy-client.cmd -Client salon-alice
```

Утилита требует чистое состояние Git, запускает статическую проверку и тесты, временно выбирает Script ID клиента, выполняет `clasp status` и проверяет доступ к проекту. Исходный `AppsScript/.clasp.json` восстанавливается даже после ошибки.

После проверки имени клиента и Script ID выполните production-загрузку:

```
.\Scripts\deploy-client.cmd -Client salon-alice -Push
```

Перед `clasp push` необходимо точно ввести имя профиля `salon-alice`. Утилита загружает только исходный код: после неё вручную обновите версию Web App deployment, выполните нужные миграции, health check и регрессию. Повторно устанавливать webhooks нужно только после изменения URL `/exec` или токена соответствующего бота.

7. Авторизация проекта

В редакторе Apps Script вручную запустите безопасную функцию чтения, например `getClientWebhookInfo`, либо будущий health check. Google запросит разрешения на таблицы, Calendar и внешние HTTP-запросы. Авторизовать должен аккаунт, от имени которого будет работать Web App.

Если Google показывает предупреждение непроверенного приложения для собственного проекта, проверьте, что вы открыли именно свой Script ID, после чего используйте разрешённый вашей организацией путь продолжения.

8. Выполнение миграций

Для актуального репозитория на чистом современном шаблоне большая часть миграций повторно-безопасна. Запускайте только миграции, присутствующие в текущем коде, и сохраняйте их результат в журнал установки.

Рекомендуемый порядок для старого или неизвестного шаблона:

1. `migrateConfigurationMessages()`;
2. `migrateConfigurationMenuIcon()`;
3. `migrateConfigurationReminderLabels()`;
4. `migratePaginationConfiguration()`;
5. `migrateSystemConfigurationSettings()`;
6. `migrateRemoveFinancialFields()`;
7. `migrateEntityNamesFromMessages()`;
8. `migrateRemoveCalendarCache()`.
9. `migrateCleanupLegacyWorkbookSchema()` — только после резервной копии legacy-таблицы.
10. Только для legacy-установки со старыми строками токенов:
`migrateBotTokensToScriptProperties()`.
11. После успешного переноса: `migrateRemoveLegacyBotTokenSettings()` — проверяет оба Script Properties и удаляет obsolete-строки из Settings.
12. `migrateCalendarResilienceMessages()` — добавляет локализованные сообщения об ошибках Calendar; функция повторно-безопасна и нужна после обновления существующей установки.

Для ротации токена откройте Apps Script → Project Settings → Script Properties и замените соответствующее значение `SMARTFLOW_CLIENT_BOT_TOKEN` или `SMARTFLOW_ADMIN_BOT_TOKEN`. После ротации переустановите webhook изменённого бота и выполните `runSmartFlowHealthCheck()`. Не записывайте новый production-токен обратно в таблицу и не включайте его в журнал установки.

На чистом актуальном шаблоне сначала сравните схему с `Docs/DATABASE_SCHEMA.md`. Не запускайте удалённые или придуманные функции. `migrateRemoveCalendarCache()` также удаляет старые cache-триггеры и создаёт часовой `syncCompletedCustomerVisitsTrigger`, если его нет.

`migrateCleanupLegacyWorkbookSchema()` восстанавливает заголовок AuditLog,

переносит старые колонки UserSessions в JSON, удаляет пустые безымянные колонки

Messages и два неиспользуемых листа. Повторный запуск безопасен, но первый запуск

необратимо удаляет устаревшую структуру, поэтому резервная копия обязательна.

9. Создание Web App deployment

1. В Apps Script выберите Deploy → New deployment.

2. Тип: Web app.
3. Execute as: пользователь, выполняющий deployment.
4. Who has access: Anyone. Telegram должен иметь возможность отправить HTTPS POST без входа в Google.
5. Выполните Deploy и скопируйте URL, заканчивающийся на `/exec`.
6. Запишите URL в Settings → `AppScriptUrl` как обычный текст без Markdown-скобок.
7. После изменения кода обновляйте существующий deployment на новую версию; один `clasp push` не всегда переключает production deployment.

10. Установка webhook

В Apps Script по очереди вручную запустите:

1. `setClientWebhook();`
2. `setAdminWebhook();`
3. `getClientWebhookInfo();`
4. `getAdminWebhookInfo();`

В информации webhook проверьте:

- Client URL заканчивается `?bot=client`;
- Admin URL заканчивается `?bot=admin`;
- `pending_update_count` не растёт постоянно;
- `last_error_message` отсутствует;
- URL использует `/exec`, а не `/dev`.

Установка webhook использует `drop_pending_updates: true`, поэтому выполняйте её осознанно: необработанные старые сообщения будут отброшены.

Повторно доставленный Telegram `update_id` безопасно игнорируется отдельно для Client Bot и Admin Bot. После обновления кода отдельная миграция не требуется: прежний максимальный ID автоматически переносится в новый ограниченный журнал Script Properties при первом webhook-запросе. Не очищайте свойства `TELEGRAM_PROCESSED_UPDATES_CLIENT` и `TELEGRAM_PROCESSED_UPDATES_ADMIN` во время нормальной работы.

11. Триггеры

Создайте time-driven trigger для `send24hAppointmentReminders`. Практичный интервал — один раз в час. Сама функция не отправляет сообщения до 08:00 и после 23:00 и учитывает `ReminderDayBefore`.

Убедитесь, что существует ровно один часовой триггер `syncCompletedCustomerVisitsTrigger`. Он фиксирует завершённые визиты и обновляет `CustomerProfiles`; он не кеширует записи и не влияет на актуальность экранов.

Не создавайте несколько одинаковых триггеров: это повышает риск повторных уведомлений и лишних квот.

12. Настройка салона через Admin Bot

1. Откройте Admin Bot под ID из `AdminTelegramIds` и нажмите Start.
2. Создайте локации.
3. Создайте услуги и длительности.

4. Создайте мастеров, привяжите их к локациям и Calendar ID.
5. Настройте недельный график и исключения мастеров.
6. При необходимости заполните неактивную строку-плейсхолдер в `RequestRecipients` для дополнительного получателя новых заявок и только после этого включите `receive_new_requests` и `active`. Всех администраторов из `AdminTelegramIds` добавлять туда не нужно.
7. Через Settings → Configuration проверьте язык, администраторов, напоминание, горизонт записи, пагинацию и часы по умолчанию.

Telegram ID мастера при создании не требуется. Его можно добавить позднее при редактировании.

Все ID из `AdminTelegramIds` автоматически получают административные уведомления. Нажатие Start не добавляет пользователя в список: доступ и уведомления появляются только после добавления ID через Configuration.

13. Приёмочная проверка

Перед ручной приёмкой запустите в редакторе Apps Script функцию `runSmartFlowHealthCheck()`. Она ничего не изменяет и возвращает структурированный отчёт. Поле `ok` должно быть `true`, `failed` — `0`.

Проверяются обязательные листы и колонки, системные настройки и часовые пояса, доступ к календарям активных мастеров, оба `webhook`, необходимые и дублирующие триггеры, а также получатели новых заявок. Отчёт не содержит токены и данные клиентов. Если проверка сообщает ошибку, исправьте её до создания тестовой заявки.

Выполните полный тест на тестовом клиенте:

1. Client Bot показывает главное меню и контакты.
2. Создаётся заявка с локацией, услугой, мастером, датой, временем, именем и телефоном.
3. Admin Bot получает заявку.
4. Администратор подтверждает один вариант, запись появляется в Appointments и Calendar.

Если Calendar временно недоступен, заявка остаётся pending: исправьте Calendar ID или права и повторно нажмите тот же вариант. Новая Appointment создаваться не должна.

5. Запись видна в Client Bot → «Мои записи» и Admin Bot → «Записи».
6. Перенос обновляет Sheets и Calendar; клиент получает карточку без повторных действий.

При ошибке Calendar перенос не подтверждается, а прежнее время и состояние напоминаний восстанавливаются.

7. Отмена удаляет связанное Calendar-событие и показывает карточку отменённой записи без кнопок.
8. Ручное Calendar-событие занимает слот только соответствующего мастера.
9. Два разных мастера могут работать одновременно.
10. Напоминание можно проверить на тестовой записи в допустимом временном окне.
11. Повторный запуск `syncCompletedCustomerVisitsTrigger` не создаёт дубль истории.

14. Типовые проблемы установки

Бот не отвечает

Проверьте `get...WebhookInfo`, URL `/exec`, актуальность `deployment`, `AuditLog` с `DOPOST_ERROR`, токен и доступ Web App «Anyone». После исправления переустановите только соответствующий `webhook`.

Admin Bot пишет «Доступ запрещён»

Проверьте точный Telegram ID и текстовый формат `AdminTelegramIds`. Значения разделяются запятыми. Не используйте точки, пробелы внутри ID или числовое округление Sheets.

Нет слотов

Проверьте активность локации, услуги и мастера, график мастера, длительность услуги, `Calendar ID`, права владельца `deployment` на календарь и часовой пояс.

Код загружен, но поведение старое

`clasp push` обновляет исходники, но production Web App может продолжать старую версию `deployment`. Создайте новую версию в `Manage deployments` и проверьте URL.

Напоминания не приходят

Проверьте `ReminderDayBefore`, триггер, допустимое время 08:00–23:00, Telegram ID клиента, статус записи и поля `reminder_24h_sent_at`. Не очищайте поля вручную без понимания сценария.

15. Безопасная передача установки салону

- передайте владельцу Google-таблицу и Apps Script проект;
- зафиксируйте, кто имеет доступ к Google и BotFather;
- сохраните Script ID, `deployment URL` и `Calendar ID` в защищённом реестре;
- не включайте токены в пользовательские инструкции;
- договоритесь о резервном копировании таблицы;
- объясните, что изменение заголовков, удаление листов и ручная правка статусов могут нарушить работу.

Официальные инструкции для iPhone

- Google Account на iPhone и iPad: <https://support.google.com/accounts/answer/6390156>
- Добавление аккаунта в Google Calendar: <https://support.google.com/calendar/answer/15619834>
- Google Calendar в стандартном календаре Apple:
<https://support.google.com/calendar/answer/99358>
- Уведомления приложений на iPhone: <https://support.apple.com/120681>